

UNIVERSIDAD NACIONAL DEL SANTA
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS E INFORMÁTICA

ALGORITMOS EVOLUTIVOS DE APRENDIZAJE

Código: 1411-2278

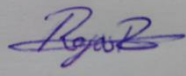
ACTIVIDAD EN AULA GRUPAL SEMANA 9
CALIFICADA (0-20)

Operadores de Selección y Cruce en Algoritmos Gen éticos

Docente: Ms. Ing. Johan Max Alexander López Heredia

Semestre: 2025-I

Duración: 35 minutos

Código	Apellidos y Nombres	Firma
202114015	Gonzales Berrocal Luchito	
202114037	Liza Guerrero Piero	
202114052	Rojas Rojas Leonardo David	
2021140	Torres Milla José Antonio	

INSTRUCCIONES GENERALES

- **Grupos:** Máximo 4-5 integrantes por grupo
- **Tiempo:** 35 minutos para completar todas las actividades
- **Material:** Solo lapicero (no calculadora, no laptop)
- **Entrega:** Al finalizar, entregar esta separata completa al docente
- **Calificación:** Evaluación grupal (0-20 puntos)

CONTEXTO: Recordando la Semana Anterior

En la **Semana 8** trabajamos con **representaciones cromosómicas** para el problema de distribución de estudiantes. Teníamos 39 alumnos que debían distribuirse en 3 exámenes (A, B, C) con 13 alumnos cada uno, buscando equilibrio en las notas.

Implementamos tres representaciones:

- **Binaria:** 117 bits ($39 \text{ alumnos} \times 3 \text{ bits c/u}$)
- **Real:** 117 valores normalizados
- **Permutacional:** Orden de 39 índices

Ahora en la **Semana 9**, estudiaremos cómo crear nuevas generaciones usando **operadores de selección y cruce**.

CASO PRÁCTICO: Sistema de Distribución Inteligente

Situación: Eres parte del equipo que desarrolla un sistema para optimizar la distribución de estudiantes. Tu algoritmo genético ya tiene una población de 6 soluciones candidatas. Ahora debes aplicar operadores de selección y cruce para generar la siguiente generación.

Población Actual (Generación 0)

Individuo	Fitness	Descripción
A	85	Muy buena distribución
B	45	Distribución regular
C	70	Buena distribución
D	20	Distribución deficiente
E	60	Distribución aceptable
F	90	Excelente distribución

ACTIVIDAD 1: SELECCIÓN POR TORNEO (8 puntos)

La **selección por torneo** elige individuos comparando pequeños grupos al azar.

Proceso: Para cada torneo, selecciona 2 individuos al azar, compara sus fitness y el mejor "gana".

Pregunta 1.1 (2 puntos)

Realiza 3 torneos de tamaño 2. Para cada torneo, indica:

- Los 2 individuos seleccionados al azar
- Sus respectivos fitness
- El ganador del torneo

Torneo 1:

Individuos seleccionados: **C** (fitness: **70**) y **F** (fitness: **90**)

Ganador: **F** ¿Por qué? **El individuo F gana porque su valor de fitness (90) es mayor que el de C (70). Esto indica que F representa una solución de distribución más óptima, por lo tanto es seleccionado como el mejor del torneo.**

Torneo 2:

Individuos seleccionados: **B** (fitness: **45**) y **E** (fitness: **60**)

Ganador: **E** ¿Por qué? **El individuo E gana porque tiene un fitness más alto (60 > 45). Esto significa que su solución es mejor evaluada que la de B en términos de calidad de distribución, lo cual justifica su selección.**

Torneo 3:

Individuos seleccionados: **A** (fitness: **85**) y **D** (fitness: **20**)

Ganador: **A** ¿Por qué? **El individuo A gana con claridad porque su fitness (85) es mucho más alto que el de D (20), lo que indica que A representa una solución muy superior. La selección por torneo favorece siempre al individuo con mejor desempeño.**

CODIGO:

```
import random

# Definir los individuos y sus fitness
individuos = {
    "A": 85,
    "B": 45,
    "C": 70,
    "D": 20,
    "E": 60,
    "F": 90
}

def seleccion_por_torneo(individuos, num_torneos=3):
    """
    Función que simula un proceso de selección por torneo.

    Parámetros:
    - individuos: Diccionario con el nombre del individuo y su fitness.
    - num_torneos: Número de torneos a realizar (por defecto 3).

    Retorna:
    - Una lista con los ganadores de cada torneo.
    """
    ganadores = []

    # Convertir los items del diccionario a una lista para usar random.sample()
    individuos_lista = list(individuos.items())

    # Para cada torneo, realizamos el proceso de selección
    for i in range(num_torneos):
        # Seleccionar aleatoriamente dos individuos
        torneo = random.sample(individuos_lista, 2)

        # Mostrar los individuos seleccionados y sus fitness
        individuo_1, fitness_1 = torneo[0]
        individuo_2, fitness_2 = torneo[1]
        print(f"Torneo {i+1}:")
        print(f"  Indiv. {individuo_1} (fitness: {fitness_1}) vs Indiv. {individuo_2} (fitness: {fitness_2})")
```

```
# Determinar el ganador del torneo
if fitness_1 > fitness_2:
    ganador = individuo_1
    print(f" Ganador: {individuo_1}\n")
else:
    ganador = individuo_2
    print(f" Ganador: {individuo_2}\n")

# Guardar el ganador del torneo
ganadores.append(ganador)

return ganadores

# Llamada a la función para simular los torneos
ganadores = seleccion_por_torneo(individuos)

# Imprimir los ganadores de los torneos
print("Ganadores de los torneos:", ganadores)
```

Pregunta 1.2 (3 puntos)

Análisis: ¿Qué individuos tienen mayor probabilidad de ser seleccionados? Explica la relación entre fitness y probabilidad de selección.

Los individuos con mayor fitness (como F, A y C) tienen más probabilidad de ser seleccionados, ya que en la selección por torneo gana el que tiene mejor rendimiento. Aunque la elección inicial es al azar, el fitness alto aumenta sus oportunidades de ser escogidos.

Pregunta 1.3 (3 puntos)

Comparación: ¿Qué ventajas tiene la selección por torneo sobre simplemente elegir siempre al mejor individuo?

La selección por torneo ofrece como principal ventaja el mantenimiento de la diversidad genética dentro de la población. A diferencia de seleccionar siempre al mejor individuo, este método da oportunidad a que otros con menor fitness también participen en el proceso evolutivo. Esto ayuda a evitar la convergencia prematura hacia soluciones subóptimas y favorece una mejor exploración del espacio de búsqueda, lo que incrementa las posibilidades de encontrar soluciones más óptimas a largo plazo.

ACTIVIDAD 2: CRUCE PMX PARA PERMUTACIONES (8 puntos)

El PMX (Partially-Mapped Crossover) es ideal para problemas donde el orden importa y no se pueden repetir elementos.

Contexto: En nuestro problema de distribución, la representación permutacional ordena a los estudiantes: posiciones [0-12] → Examen A, [13-25] → Examen B, [26-38] → Examen C.

Ejemplo de Cruce PMX

Padres (8 elementos para simplificar):

Padre 1:	1	2	3	4	5	6	7	8
Padre 2:	8	7	6	5	4	3	2	1

Puntos de cruce: Entre posiciones 3 y 6 (segmento resaltado en rojo)

Pregunta 2.1 (3 puntos)

Paso 1: Identifica el mapeo del segmento intercambiado.

Segmento del Padre 1: 4-5 ↔ Segmento del Padre 2: 5 - 4

Mapeo generado: 4 (P1) ↔ 5 (P2)
5 (P1) ↔ 4 (P2)

Pregunta 2.2 (5 puntos)

Paso 2: Construye el Hijo 1 aplicando PMX.

a) Copia el segmento del Padre 2 al Hijo 1:

Hijo 1:	-	-	-	<u>5</u>	<u>4</u>	-	-	-
----------------	---	---	---	----------	----------	---	---	---

b) Completa las posiciones restantes usando el mapeo y el Padre 1:

Posición 0: Valor del Padre 1 = 1, ¿está en el segmento? NO
 Si está, usar mapeo: -, sino copiar directamente: -

Posición 1: Valor del Padre 1 = 2, ¿está en el segmento? NO
 Si está, usar mapeo: -, sino copiar directamente: -

Posición 2: Valor del Padre 1 = 3, ¿está en el segmento? NO
 Si está, usar mapeo: -, sino copiar directamente: -

Repetimos el mismo patrón para completar todo el Hijo 1, ningún valor reemplazado está en el segmento

c) **Hijo 1 completo:**

Hijo 1:	<u>1</u>	<u>2</u>	<u>3</u>	<u>5</u>	<u>4</u>	<u>6</u>	<u>7</u>	<u>8</u>
----------------	----------	----------	----------	----------	----------	----------	----------	----------

Implementación en python:

```

# Padres del ejemplo (con 8 elementos)
padre_1 = [1, 2, 3, 4, 5, 6, 7, 8]
padre_2 = [8, 7, 6, 5, 4, 3, 2, 1]

# Puntos de cruce (índices)
inicio_cruce = 3
fin_cruce = 6 # Segmento de cruce: posiciones 3, 4, 5

# Paso 1: Copiar el segmento del Padre 2 al Hijo 1
hijo_1 = [-1] * len(padre_1)
hijo_1[inicio_cruce:fin_cruce] = padre_2[inicio_cruce:fin_cruce]

# Crear el mapeo entre los genes intercambiados
mapeo = {}
for i in range(inicio_cruce, fin_cruce):
    mapeo[padre_1[i]] = padre_2[i]
    mapeo[padre_2[i]] = padre_1[i] # mapeo bidireccional

# Paso 2: Completar el resto del hijo con los genes del Padre 1
for i in range(len(padre_1)):
    if i >= inicio_cruce and i < fin_cruce:
        continue # Saltar el segmento de cruce

    gen = padre_1[i]

    # Resolver conflictos: si el gen ya está en el hijo, usar el mapeo
    while gen in hijo_1[inicio_cruce:fin_cruce]:
        gen = mapeo[gen] # usar el mapeo hasta encontrar uno válido

    hijo_1[i] = gen

# Mostrar resultados
print("Padre 1:", padre_1)
print("Padre 2:", padre_2)
print("Hijo 1: ", hijo_1)

```

ACTIVIDAD 3: ANÁLISIS COMPARATIVO (4 puntos)**Pregunta 3.1 (2 puntos)**

Selección por Ruleta vs Torneo: ¿En qué situación preferirías usar selección por torneo en lugar de selección por ruleta?

Preferiría usar selección por torneo cuando trabajo con poblaciones donde la distribución de fitness es muy desigual, o cuando hay riesgo de que unos pocos individuos con fitness muy alto dominen el proceso evolutivo, como en problemas de optimización difíciles o multimodales.

La selección por torneo es menos sensible a las diferencias extremas en fitness que la ruleta, lo cual ayuda a mantener mayor diversidad genética en la población y reduce la probabilidad de convergencia prematura. Además, es más sencilla de implementar, escalable y no requiere normalizar los valores de fitness, lo que la hace más robusta en ambientes donde el fitness puede fluctuar o tener valores negativos.

Pregunta 3.2 (2 puntos)

Aplicación práctica: Si tuvieras que resolver un problema del viajante de comercio (TSP) con 20 ciudades, ¿qué representación cromosómica y qué operador de cruce usarías? Justifica tu respuesta.

Representación: Permutacional **Cruce:** PMX

Justificación:

En el problema del TSP, cada ciudad debe ser visitada exactamente una vez, por lo tanto la mejor representación cromosómica es una permutación de enteros. Esta codificación garantiza rutas válidas sin ciudades repetidas ni omitidas.

Para mantener esta validez durante el cruce, usaría el operador PMX, ya que está diseñado específicamente para trabajar con permutaciones. PMX preserva el orden relativo y la posición de los genes, y resuelve las colisiones mediante un mapeo que asegura que no haya duplicados. Esto lo hace ideal para problemas como el TSP, donde el orden y la unicidad son críticos.

GLOSARIO

Algoritmo Genético (AG)

Técnica de optimización inspirada en la evolución natural que usa operadores como selección, cruce y mutación.

Cromosoma

Representación codificada de una solución al problema (equivale a un individuo).

Fitness Valor numérico que indica qué tan buena es una solución (aptitud).

Generación

Conjunto de individuos en un momento específico del algoritmo.

PMX (Partially-Mapped Crossover)

Operador de cruce específico para representaciones permutacionales que mantiene la validez de la permutación.

Población

Conjunto de todas las soluciones candidatas en una generación.

Presión Selectiva

Intensidad con la que se favorece a los individuos más aptos durante la selección.

Representación Permutacional

Codificación donde la solución es un ordenamiento de elementos sin repetición.

Selección por Torneo

Método que elige padres comparando pequeños grupos de individuos seleccionados al azar.

Selección por Ruleta

Método que asigna probabilidades de selección proporcionales al fitness de cada individuo.

¡Éxito en la actividad!

Recuerden: La evolución artificial requiere los mismos principios que la natural: selección y recombinación.

